JoBot - Comprehensive Refactor, Production & Release Readiness Plan

Repository: https://github.com/aryansinghnagar/JoBot
Date: 2026-08-15
Basis: JoBot repository audit, repository planning documents, `.agents/agents.md`, current source tree, tests/CI/release configuration, and comparative research into open-source and proprietary agent/job-search systems.

Executive Summary

JoBot already has the architecture of an ambitious local-first agent platform: an async execution fabric, task graph concepts, application-state protocol, saga orchestration, provider-neutral LLM routing, browser automation, encrypted storage, OS keyring integration, policy controls, memory, tracing, evaluations, adapters, GUI, CI, SBOM generation, and PyPI publication.

The central recommendation is therefore **not to rewrite JoBot and not to immediately add another large collection of capabilities**.

The next milestone should be:

> **Make one end-to-end job application durable, verifiable, recoverable, observable, secure, and reproducible under failure.**

Only after that foundation is proven should JoBot aggressively expand to additional job boards, networking, market intelligence, large-scale application campaigns, self-improvement, and broader agent capabilities.

Core production invariant

```
NO ACTION WITHOUT A STATE
NO STATE WITHOUT AN EVENT
NO COMPLETION WITHOUT VERIFICATION
NO SIDE EFFECT WITHOUT POLICY
NO RETRY WITHOUT IDEMPOTENCY
NO LONG RUN WITHOUT CHECKPOINT
NO MEMORY WITHOUT PROVENANCE
NO AUTONOMY WITHOUT MEASUREMENT
```

---

1. Current-State Assessment

1.1 What JoBot already contains

The repository currently contains:

- CLI and GUI surfaces
- Python async execution components
- task graph
- application submission pipeline
- saga orchestration
- job-board adapters
- discovery/scrapers
- document tailoring and PDF generation
- ATS scoring
- cover-letter generation
- interview preparation
- LLM routing/providers
- memory/vector storage
- security and PII masking
- policy engine
- scheduler
- browser/stealth layer
- plugin infrastructure
- tracing/alerts
- eval harness
- extensive unit/integration tests
- Docker configuration
- GitHub Actions CI
- CodeQL
- Dependabot
- SBOM/provenance generation
- PyPI publishing

The repository README describes JoBot as a local-first, privacy-preserving job-application operating system with Patchright browser automation, SQLite WAL storage, Fernet encryption, OS keyring integration, and provider-neutral model routing.

The current Merge Plan similarly positions JoBot as a sophisticated architectural chassis onto which capability modules should be grafted rather than copied wholesale.

1.2 Current maturity estimate

Area	Current	Target
Domain model	7/10	9/10
Job adapters	6/10	9/10
Application workflow	7/10	9.5/10
Durable execution	4/10	9.5/10
Task graph	3/10	9/10
State persistence	6/10	9/10
Idempotency	6/10	9.5/10
Saga/compensation	5/10	9/10
Browser automation	5/10	9/10
Security	6/10	9/10
Policy/governance	5/10	9/10
LLM routing	6.5/10	9/10
Memory	4/10	8.5/10
Observability	5/10	9/10
Evals	5/10	9/10
GUI/control plane	5/10	9/10
CI	7/10	9.5/10
Packaging/release	4/10	9.5/10
Documentation	6/10	9/10
Overall production readiness	~5/10	9/10

These are engineering-readiness estimates, not measurements from a running production environment.

---

2. Strategic Recommendation

Do not immediately optimize for volume

The existing roadmap emphasizes adding capabilities such as JobSpy, additional job boards, resume tailoring, question answering, and application automation.

The safer sequencing is:

```
Durable execution
      ->
State correctness
      ->
Browser reliability
      ->
Verification/evidence
      ->
AI reliability/evals
```

```
      ->
Production release
      ->
Capability expansion
      ->
High-volume autonomy
```

The key reason is that a high-throughput agent magnifies correctness problems. A duplicated or unverified application at scale is materially worse than a missing feature.

---

3. Critical Architecture Defects

3.1 TaskGraph is not yet truly durable or atomic

The current task graph stores tasks in an in-memory dictionary and claims tasks by mutating object state.

That does not provide durable multi-worker coordination.

Target

Create first-class entities:

```
Task
TaskAttempt
TaskLease
TaskEvent
TaskArtifact
TaskDependency
```

Use persistent status:

```
PENDING
READY
CLAIMED
RUNNING
WAITING
RETRYING
VERIFYING
COMPLETED
FAILED
QUARANTINED
CANCELLED
UNKNOWN
```

Implement atomic claiming at the database level using conditional updates/transactions.

Exit criterion

Multiple workers cannot claim the same task; a killed worker's lease eventually expires and the task becomes recoverable.

3.2 Introduce an event ledger

Add:

```
events
------
event_id
aggregate_type
aggregate_id
event_type
event_version
payload
actor
correlation_id
causation_id
created_at
```

Example events:

```
GoalCreated
TaskCreated
TaskClaimed
TaskStarted
ToolInvoked
ToolCompleted
ToolFailed
ApprovalRequested
ApprovalGranted
ApprovalRejected
BrowserSessionStarted
BrowserSessionLost
ApplicationPrepared
ApplicationSubmitted
ApplicationVerified
ApplicationFailed
BudgetExceeded
WorkerDisconnected
RunResumed
```

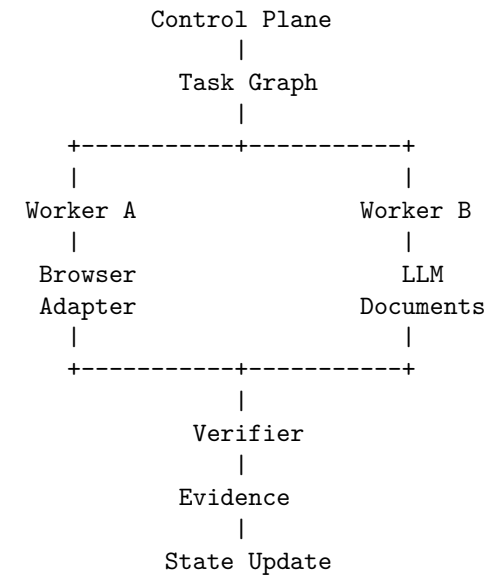
```
RunQuarantined
MemoryUpdated
EvalCompleted
```

This becomes the foundation for auditability, replay, timelines, debugging, analytics, and recovery.

---

4. State Machine and Execution Separation

The architecture should become:



Avoid allowing arbitrary agents to directly mutate durable state and directly execute external side effects.

Instead:

```
Agent proposes action
       ->
Policy
       ->
Execution adapter
       ->
Effect record
       ->
Verification
       ->
State transition
```

5. Saga and Effect Ledger

The current saga abstraction is directionally correct but compensation is currently closer to state bookkeeping than true external-effect compensation.

Introduce:

```
ExternalEffect
--------------
effect_id
task_id
application_id
effect_type
idempotency_key
request_hash
started_at
completed_at
status
external_reference
verification_state
compensation_state
```

For reversible effects, compensation can undo the action.

For irreversible effects:

```
irreversible effect
       ->
record effect
       ->
verify
       ->
quarantine on ambiguity
       ->
prevent replay
```

The invariant is:

> A recovery process must never replay an external side effect merely because local state is ambiguous.

6. Durable Human Approval

The existing `PENDING_APPROVAL` path should become a first-class persistent entity.

```
ApprovalRequest
---------------
id
task_id
action
risk_level
proposed_arguments
evidence
policy_reason
expires_at
requested_at
decided_at
decided_by
decision
modified_arguments
```

Decisions:

```
APPROVE
EDIT
REJECT
DEFER
CANCEL
```

The GUI, CLI, and future MCP interface should consume the same approval model.

---

7. Risk and Trust Model

Replace a mostly application-specific policy model with a generalized action-risk model.

```
Risk = f(
  action,
  target,
  reversibility,
  credentials,
  external_side_effect,
  personal_data,
  financial_cost,
  volume,
```

```
  confidence,
  trust_level
)
```

Suggested tiers:

Risk	Example	Default
R0	Local read	Auto
R1	Public job scrape	Auto
R2	Resume generation	Auto
R3	Application form preparation	Auto/Draft
R4	Save application	Policy dependent
R5	Submit application	Approval
R6	Recruiter outreach	Approval
R7	Credential modification	Approval
R8	Irreversible/high-impact external action	Human only

Trust should be scoped to the site, tool, skill, or action rather than becoming one global autonomy switch.

---

8. Browser Automation Subsystem

Browser automation should become an explicit subsystem:

```
BrowserSessionManager
  |
  +-- BrowserPool
  +-- ProfileStore
  +-- SessionPersistence
  +-- Navigation
  +-- NamedActions
  +-- SelectorRegistry
  +-- SelectorHealing
  +-- ScreenshotEvidence
  +-- CAPTCHA Boundary
  +-- RateLimitDetector
  +-- SiteHealth
  +-- Recovery
```

Every risky browser action should capture:

```
Before screenshot
Relevant DOM/form snapshot
Action
```

```
Arguments
After screenshot
Result
Verification
Trace ID
Application ID
```

Important reliability principle

Site blocking should lead to:

```
SITE_BLOCKED
    ->
Circuit breaker
    ->
Health incident
    ->
Quarantine
    ->
Alternate source / human action
```

Do not build the product around indefinite attempts to circumvent platform protections.

---

9. Campaign Runner Refactor

The current continuous campaign runner contains hardcoded titles, portals, loop behavior, and short sleeps and is not a durable campaign coordinator.

Replace it with:

```
Campaign
   |
Goal
   |
Discovery Tasks
   |
Ranking Tasks
   |
Application Preparation Tasks
   |
Approval Tasks
   |
Submission Tasks
   |
Verification Tasks
```

```
   |
Outcome Tracking
```

Persist campaign state.

A campaign must resume after process death without duplicating work.

---

10. LLM Router v2

The current model router already provides provider abstraction, fallback chains, cost awareness, task overrides, health checking, and local providers.

Make it production-grade by moving spend accounting into the durable control plane.

Introduce:

```
llm_calls
budgets
budget_reservations
provider_health
model_capabilities
routing_decisions
```

Capability-aware routing

```yaml
task: resume_tailoring
requirements:
  structured_output: true
  long_context: true
  reasoning: medium
  latency: medium
  cost: low
```

Routing should consider:

```
Capability
Quality
Cost
Latency
Availability
Historical success
```

rather than provider order alone.

---

11. Prompt Management

Prompts should be versioned like source code.

Suggested structure:

```
prompts/
  application/
    fit_evaluation/
      v1.yaml
      v2.yaml
    resume_tailoring/
    cover_letter/
    question_answering/
    interview/
```

Record for every model call:

```
prompt_id
prompt_version
model
provider
temperature
schema_version
tool_version
profile_version
```

This makes prompt changes measurable and rollbackable.

---

12. Candidate Truth System

This should become one of JoBot's core differentiators.

Create:

```
CandidateFact
-------------
fact_id
category
value
source
evidence
confidence
valid_from
valid_until
```

Example:

```
skill: Python
source: Resume
confidence: 1.0

skill: Kubernetes
source: project artifact
confidence: 0.82

experience: distributed inference pipeline
source: project evidence
confidence: 0.91
```

Generated application material should be grounded in candidate facts.

The LLM should be allowed to propose facts, not silently mutate authoritative profile truth.

---

13. Layered Memory

Explicitly separate:

```
Hot
  Current task/application

Warm
  Candidate preferences and active search

Semantic
  Stable candidate facts

Episodic
  What happened in application X

Procedural
  How a site workflow works

Historical
  Past applications/outcomes

External
  Company/market knowledge
```

Every important memory entry needs provenance and confidence.

---

14. Evidence System

For jobs:

```
JobPosting
  source URL
  source timestamp
  raw posting
  normalized representation
  extraction version
```

For applications:

```
Application
  original JD
  fit score
  tailored resume
  cover letter
  submitted values
  screenshots
  confirmation
  verification
  outcome
```

Evidence should be accessible from the GUI and trace.

---

15. Resume and Cover-Letter Pipeline

Target:

```
Job Description
      |
JD Parser
      |
Fit Evaluator
      |
Evidence-backed candidate facts
      |
Tailoring planner
      |
Draft
      |
Independent reviewer
      |
Revision
      |
```

```
PDF compilation
      |
PDF text extraction
      |
ATS verification
      |
Visual verification
      |
Artifact
```

A reviewer should be independent enough to catch unsupported claims, keyword stuffing, formatting failure, and contradictions.

---

16. Multi-Stage Job Matching

Do not send every discovered job through an expensive model.

Use:

```
Stage 1 - Deterministic filtering
  location
  visa
  salary
  title
  employment type

Stage 2 - Cheap semantic matching
  lexical/embedding score

Stage 3 - Structured LLM evaluation

Stage 4 - Deep company/job research
  only for shortlisted jobs

Stage 5 - Recommendation
```

This controls LLM cost and improves throughput.

---

17. Job Quality and Fraud Detection

Add a job-risk classifier covering:

```
Company legitimacy
Domain mismatch
```

```
Salary anomalies
Duplicate posting
Recruiter authenticity
Application-domain mismatch
Suspicious payment requests
Credential requests
Malicious embedded instructions
Prompt injection
```

External job descriptions must be treated as untrusted content.

---

18. Prompt-Injection Boundary

Establish a strict boundary:

```
External job content
       |
UNTRUSTED
       |
Parse / Extract
       |
Never execute instructions from source content
```

Add:

- HTML sanitization
- instruction classification
- tool-call isolation
- URL allowlisting
- secret isolation
- policy enforcement before tool use
- adversarial evals

---

19. MCP Integration

Expose JoBot as an optional MCP server while keeping MCP out of the core domain model.

Potential tools:

```
search_jobs
get_job
rank_jobs
get_candidate_profile
```

```
generate_resume
get_application
prepare_application
request_application_approval
get_application_evidence
get_search_analytics
```

MCP becomes an interoperability boundary for external agents, not JoBot's internal execution engine.

---

20. GUI as Control Plane

The existing Tauri application should become the operational control plane.

Home

```
Active work
Pending approvals
Failures
Daily applications
Costs
Top matches
```

Task

```
Status
Owner
Attempts
Dependencies
Current phase
Evidence
Logs
Cost
```

Application

```
Job
Fit
Resume
Cover letter
Questions
Submission state
Screenshots
Verification
```

Approval

```
WHAT
WHY
RISK
EVIDENCE

[Approve] [Edit] [Reject] [Defer]
```

Incident

```
What happened
Timeline
Affected applications
Root cause
Current mitigation
Recommended fix
```

---

21. Observability

Use a standard trace model.

Trace hierarchy:

```
Goal
  |
Task
  +-- Model call
  +-- Tool call
  +-- Browser action
  +-- Policy evaluation
  +-- Approval
  +-- Verification
  +-- Artifact
```

Every trace should include:

```
trace_id
run_id
goal_id
task_id
application_id
worker_id
provider
model
prompt_version
```

```
policy_version
adapter_version
```

Local-first storage can coexist with optional OpenTelemetry and Langfuse exports.

---

22. Evaluation Platform

Make evals a release gate.

Required suites:

Capability

- job discovery
- parsing
- ranking
- resume tailoring
- form filling
- submission
- verification

Reliability

Measure:

```
pass@1
pass@N
median duration
retry count
intervention rate
silent failure rate
```

Security

Test:

```
Prompt injection
Secret exfiltration
Malicious URLs
Fake job postings
Credential requests
Destructive tool requests
Malicious plugins
```

Long-horizon

Example:

```
Find 20 relevant ML jobs
 ->
Rank
 ->
Shortlist
 ->
Prepare 3 applications
 ->
Request approvals
 ->
Submit
 ->
Verify
 ->
Record outcomes
```

Failure injection

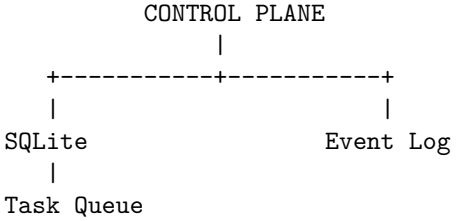
Inject:

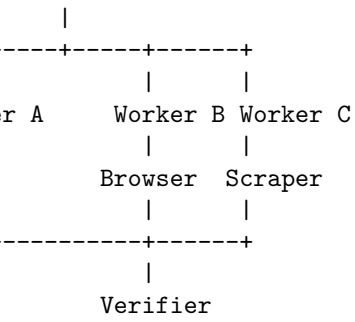
```
Browser crash
Network failure
Provider timeout
Rate limit
DB lock
Process kill
Worker disconnect
Malformed form
CAPTCHA
Stale session
```

---

23. Production Worker Architecture

Initial production architecture:





Initially this can remain local-first.

Later, the durable interfaces should permit Postgres/queue/remote workers without redesigning the domain.

---

24. Sandbox Execution

Support execution isolation:

```
Local execution
     ->
Restricted subprocess
     ->
Container sandbox
     ->
Remote sandbox
```

Unknown or untrusted plugins and arbitrary generated code should not execute with unrestricted access to candidate credentials or the host filesystem.

---

25. Plugin Security

Plugin installation should be:

```
Manifest
 ->
Source/hash verification
 ->
Permissions
 ->
Dependency scan
 ->
Sandbox capability
```

```
 ->
Install
 ->
Health check
```

Plugins should declare explicit permissions for:

```yaml
permissions:
  filesystem:
    - candidate-profile
  network:
    - api.example.com
  browser:
    - linkedin
  secrets:
    - none
```

Default policy should be deny-by-default.

---

26. Job Lifecycle and Entity Model

Introduce richer normalized entities.

Job lifecycle

```
DISCOVERED
NORMALIZED
DEDUPLICATED
ENRICHED
MATCHED
SHORTLISTED
EXPIRED
REJECTED
APPLIED
```

Company

```
Company
CompanyDomain
CompanyAlias
CompanyLocation
CompanyEmployee
CompanyRecruiter
CompanyJob
```

Normalize company aliases to one canonical entity.

---

27. Outcome Learning

The long-term differentiator should be learning from outcomes.

Track:

```
Job
Application
Resume variant
Skills
Match score
Application source
Application timing
Networking activity
Interview
Offer/Rejection
```

Then learn:

```
Which roles generate interviews?
Which companies respond?
Which resume variants perform?
Which skills improve outcomes?
Which application sources work?
```

This is more valuable than application volume alone.

---

28. Career Intelligence Layer

Long term:

```
Candidate
   |
Skill Graph
   |
Market Graph
   |
Job Graph
   |
Outcome Graph
```

This enables questions such as:

```
What should I apply to?
Why am I not getting interviews?
```

```
What skill should I learn next?
Which companies respond to me?
Which roles offer the highest expected return?
Which resume version performs best?
```

That is the path from a job automation tool to a career operating system.

---

29. Self-Improvement

The improvement loop should be bounded:

```
Production failure
      |
Failure classifier
      |
Identify gap:
  skill?
  tool?
  policy?
  prompt?
  memory?
  decomposition?
  verifier?
      |
Improvement proposal
      |
Sandbox branch
      |
Eval
      |
Baseline comparison
      |
Gate
      |
Promote or discard
```

Never permit unrestricted production self-modification.

---

30. Skill Extraction

Successful trajectories can become reusable skills:

```
Trajectory
   |
Generalize
   |
Skill Candidate
   |
Review/Eval
   |
Skill Registry
```

Example:

`workday_application_v1`

can capture:

- navigation
- selectors
- question classes
- common failure modes
- recovery strategy
- verification behavior

Each skill should have its own test corpus.

---

31. Repository Refactor Direction

Gradually converge toward:

```
src/jobot/
    core/
        events/
        state/
        tasks/
        workflows/
        errors/

    control/
        goals/
        approvals/
        budgets/
        trust/
        policies/
        incidents/

    execution/
```

```
    workers/
    leases/
    sandbox/
    browser/
    tools/

ai/
    router/
    providers/
    prompts/
    profiles/
    evaluation/

memory/
    semantic/
    episodic/
    procedural/
    retrieval/

career/
    matching/
    scoring/
    market/
    networking/
    interview/

applications/
    state_machine/
    preparation/
    submission/
    verification/

adapters/
    ats/
    boards/
    browser/

documents/
    resume/
    cover_letter/
    pdf/
    ats/

observability/
    tracing/
    metrics/
```

```
        events/

    plugins/
```

Do this incrementally. Do not perform a single massive directory rewrite.

---

32. Release Engineering

The repository already has CI, CodeQL, Dependabot, SBOM/provenance, and PyPI publishing. That is a good base.

However, the current PyPI pipeline builds/uploads on published releases and there are currently no GitHub releases.

Version authority

There is a current inconsistency between the package version declared in `pyproject.toml` and the release-manager test expectation.

Establish one authoritative version source:

```
pyproject.toml
    |
Package
    |
CLI --version
    |
GUI version
    |
Git tag
    |
Docker tag
    |
Release metadata
```

---

33. CI Matrix

Recommended Python versions:

```
3.11
3.12
3.13
```

Supported platforms should be explicitly classified as Tier 1, Tier 2, or community-supported rather than all being treated equally.

Recommended test layers:

```
Fast PR suite
Full main-branch suite
Nightly browser suite
Nightly integration/eval suite
Release candidate suite
```

---

34. Release Pipeline

```
PR
 ->
Lint
 ->
Format
 ->
Mypy
 ->
Unit Tests
 ->
Integration Tests
 ->
Security Tests
 ->
Eval Suite
 ->
Package Build
 ->
GUI Build
 ->
Docker Build
 ->
SBOM
 ->
Artifact Signing/Provenance
 ->
Release Candidate
 ->
Smoke Test
 ->
GitHub Release
 ->
PyPI
 ->
```

```
Container Registry
 ->
Desktop Artifacts
```

Release channels:

```
nightly
alpha
beta
rc
stable
```

---

35. Release Gates

A release cannot ship unless:

Functional

- Unit tests pass
- Critical integration tests pass
- Critical evals pass
- State-machine transitions are covered
- Adapter contracts pass

Reliability

- Crash/recovery test passes
- Duplicate submission test passes
- Browser reconnect test passes
- Provider fallback test passes
- Approval/resume test passes

Security

- CodeQL
- Dependency audit
- Secret scan
- Plugin permission tests
- Prompt injection suite
- Credential leakage suite

Packaging

- Wheel installation
- Source distribution installation

- Docker smoke test
- GUI build
- `jobot doctor`
- Database migration
- Upgrade from previous release

---

36. Database Migrations

Replace ad-hoc additive migration logic with versioned migrations:

```
schema_migrations
-----------------
version
applied_at
checksum
```

Example:

```
001_initial
002_events
003_task_leases
004_approvals
005_budget
006_effects
007_memory_provenance
```

CLI:

```
jobot db status
jobot db migrate
jobot db backup
jobot db restore
jobot db verify
```

---

37. Backup and Recovery

Local-first reliability requires backup.

Add:

```
jobot backup
jobot restore
jobot verify-backup
```

Backup:

```
SQLite state
Encrypted profile
Application artifacts
Memory
Configuration metadata
```

Do not indiscriminately include temporary browser caches or unnecessary secrets.

---

38. `jobot doctor` as a Release-Critical Capability

Expand the existing doctor command to test:

```
Runtime
  Python
  OS
  SQLite
  filesystem

Security
  keyring
  encrypted store
  permissions

Browser
  Patchright
  Chromium
  profiles

Documents
  LaTeX
  PDF renderer
  pdftotext

AI
  configured providers
  health checks

Adapters
  adapter registry
  capability health

Control plane
  migrations
```

```
  event log
  queue
  worker

Release
  package metadata
  compatibility
```

Also support:

```
jobot doctor --json
```

---

39. Secret Management

Keep OS keyring and encrypted storage, but add:

- secret type registry
- rotation support
- access audit
- aggressive redaction
- no secrets in events
- no passwords in prompts
- scoped tool access
- browser session auth where possible

Production should fail closed if critical encryption/secrets requirements are missing.

---

40. Application Question Answering

Use:

```
Question
  |
Classifier
  |
Known candidate fact?
  | yes
  +----> use grounded fact
  |
  no
  |
Policy
  |
```

```
LLM proposal
  |
Grounding verifier
  |
Approval if required
```

Classify:

```
FACTUAL
PREFERENCE
ELIGIBILITY
LEGAL
SENSITIVE
FREE_TEXT
UNKNOWN
```

Never allow model invention of material candidate facts.

---

41. Application Submission Verification

Do not equate form submission with successful submission.

Use:

```
Submit
  |
Observe
  |
Confirmation detection
  |
Extract confirmation ID
  |
Screenshot
  |
Persist evidence
  |
Reconcile portal state
  |
VERIFIED
```

Ambiguous state should be:

```
SUBMISSION_UNKNOWN
```

not `SUBMITTED`.

---

42. Unknown State as a First-Class Concept

Many agent failures come from binary success/failure models.

JoBot needs:

```
submission_unknown
verification_unknown
provider_unknown
browser_unknown
company_identity_unknown
job_expiry_unknown
email_signal_ambiguous
```

Unknown should trigger reconciliation, quarantine, or human review rather than blind retry.

---

43. Feature Integration Matrix

Feature	Primary inspiration	Priority
Durable task execution	LangGraph / Temporal	P0
Persistent checkpoints	LangGraph	P0
Human approval interrupts	LangGraph	P0
Layered memory	Letta	P0
Typed tools	PydanticAI	P0
Agent profiles	PydanticAI / Claude Code	P0
Reviewer agents	ai-job-search	P0
Application saga/effects	Saga/Temporal patterns	P0
Browser evidence	OpenHands	P0
Sandbox	OpenHands / E2B	P1
Model routing	LiteLLM	P1
Prompt versioning	Langfuse	P1
Trace/eval platform	OpenTelemetry / Langfuse	P0
Temporal knowledge	Graphiti	P2
MCP	MCP ecosystem	P1
Plugin permissions	OpenClaw	P1
Scheduler	OpenClaw / agent runtimes	P1
Skill extraction	Hermes-like systems	P2
Bounded self-improvement	DSPy / agent eval practice	P2
Multi-board scraping	JobSpy	P1
Career scoring	career-ops	P0
Resume verification	ai-job-search	P0
Job fraud detection	Jobright	P1
Networking	Simplify / Teal patterns	P1

Feature	Primary inspiration	Priority
Application tracking	Simplify / Teal / JobSync	P1
Market intelligence	Jobright / career platforms	P2

---

44. Open-Source Projects to Study

MadsLorentzen/ai-job-search

https://github.com/MadsLorentzen/ai-job-search

Study:

- profile-first architecture
- job discovery
- ranking
- resume tailoring
- cover letters
- interview prep
- follow-up workflow
- application outcome tracking
- reviewer-agent workflow
- ATS PDF checks

Best lesson:

> Treat the complete job search as one connected workflow rather than unrelated AI utilities.

JobSpy

https://github.com/speedyapply/JobSpy

Study:

- multi-board scraping
- normalization
- source aggregation

Best use in JoBot:

> Integrate behind the adapter layer rather than making JobSpy the application architecture.

Career-ops

https://github.com/santifer/career-ops

Study:

- capability registry
- job scoring
- company research
- networking
- CV tooling
- portal scanning
- command-center concepts

JobSync

https://github.com/Gsync/jobsync

Study:

- self-hosted dashboard
- scheduled discovery
- AI matching
- application tracking
- MCP exposure

OpenHands

https://github.com/All-Hands-AI/OpenHands

Study:

- sandboxed agent execution
- runtime abstraction
- tool isolation
- long-running agent workflows

LangGraph

https://github.com/langchain-ai/langgraph

Study:

- durable execution
- checkpoints
- interrupts
- human-in-the-loop
- explicit graph state

Letta

https://github.com/letta-ai/letta

Study:

- persistent memory
- memory management
- stateful agents

PydanticAI

https://github.com/pydantic/pydantic-ai

Study:

- typed tools
- structured model outputs
- dependency injection
- agent testability

---

45. Proprietary Product Features Worth Reproducing

Simplify

https://simplify.jobs/

Useful concepts:

- canonical candidate profile
- personalized matching
- tailored resume
- autofill
- application tracking
- networking

Teal

https://www.tealhq.com/

Useful concepts:

- job tracker
- application checklist
- keyword analysis
- contact management
- follow-up organization

Jobright

https://jobright.ai/

Useful concepts:

- personalized matching
- proactive job discovery
- networking
- career intelligence
- job quality/fraud signals
- career coaching

Do not copy product UX or proprietary implementation; reproduce the underlying product capabilities using JoBot's own architecture.

---

46. Phase Roadmap

Phase 0 - Baseline and Freeze

Target: 1-2 weeks

Deliver:

- architecture inventory
- dependency inventory
- runtime matrix
- test baseline
- eval baseline
- security baseline
- performance baseline
- production-readiness scorecard

Do not add major product capabilities.

Phase 1 - Durable Execution Core

Target: 2-4 weeks

Implement:

- persistent tasks
- leases
- heartbeats
- attempts
- events
- checkpoints
- retry policies
- quarantine
- cancellation

- recovery

Exit criterion:

> Kill a worker during each major execution phase and resume correctly.

Phase 2 - Application State Correctness

Target: 2-3 weeks

Implement:

- formal state machine
- event ledger
- effect ledger
- idempotency
- verification
- unknown states
- durable approval
- reconciliation

Exit criterion:

> Interrupted applications never create duplicate external submissions.

Phase 3 - Browser Reliability

Target: 3-5 weeks

Implement:

- browser pool
- session lifecycle
- named actions
- evidence capture
- selector registry
- selector healing
- portal health
- recovery
- CAPTCHA boundary

Exit criterion:

> Mock ATS and supported real workflows survive injected browser/network failures.

Phase 4 - AI Reliability

Target: 2-4 weeks

Implement:

- typed LLM contracts
- structured output
- prompt registry
- prompt versioning
- capability-aware routing
- cost ledger
- provider health
- fallback policies
- independent reviewers
- candidate truth system

Exit criterion:

> No critical eval produces unsupported candidate claims.

Phase 5 - Evaluation Platform

Target: 2-4 weeks

Implement:

- datasets
- trajectory recorder
- eval runner
- baseline comparator
- regression detector
- security corpus
- failure corpus

Exit criterion:

> Every release can demonstrate whether agent quality improved or regressed.

Phase 6 - Control Plane / GUI

Target: 3-5 weeks

Implement:

- dashboard
- task inspector

- approval inbox
- evidence viewer
- trace viewer
- cost dashboard
- incident dashboard
- worker status
- career funnel

Phase 7 - Capability Expansion

Add in priority order:

1. JobSpy integration
2. ATS/board expansion
3. better job ranking
4. company normalization
5. interview preparation
6. follow-up workflows
7. networking/outreach
8. email synchronization
9. salary/market intelligence
10. fraud detection
11. MCP
12. sandbox
13. browser skills

Phase 8 - Stable Release

Release channels:

```
nightly
alpha
beta
rc
stable
```

Suggested product milestone:

```
0.x  - experimental
0.9  - release candidate
1.0  - stable supported workflow
```

---

47. P0 Backlog

Correctness

- ☐ Persistent task queue
- ☐ Atomic task leasing
- ☐ Task attempts
- ☐ Event ledger
- ☐ Worker heartbeat
- ☐ Checkpoint/resume
- ☐ Durable approval
- ☐ Unknown state model
- ☐ State-machine validation
- ☐ Effect ledger
- ☐ Submission reconciliation
- ☐ Idempotency audit

Security

- ☐ Prompt-injection boundary
- ☐ Credential isolation
- ☐ Secret redaction
- ☐ Plugin permissions
- ☐ Browser profile isolation
- ☐ Sandbox execution
- ☐ Threat model
- ☐ Security eval suite

Verification

- ☐ Independent verifier
- ☐ PDF visual verification
- ☐ PDF text-layer verification
- ☐ Submission evidence
- ☐ Trace/evidence correlation
- ☐ Failure injection

Release

- ☐ Single version authority
- ☐ Versioned DB migrations
- ☐ Clean install test
- ☐ Wheel/sdist test
- ☐ Docker smoke test
- ☐ GUI build
- ☐ RC pipeline

- ☐ Upgrade test
- ☐ Rollback procedure

---

48. P1 Backlog

- ☐ JobSpy
- ☐ ATS expansion
- ☐ Company entity graph
- ☐ Better job matching
- ☐ Career scoring
- ☐ Interview preparation
- ☐ Follow-ups
- ☐ Email sync
- ☐ Networking
- ☐ Salary intelligence
- ☐ MCP
- ☐ OpenTelemetry
- ☐ Optional Langfuse
- ☐ Sandbox
- ☐ Browser skill registry
- ☐ Job fraud detection
- ☐ Application analytics
- ☐ Career funnel analytics

---

49. P2 Strategic Moat

- ☐ Outcome-based job matching
- ☐ Career opportunity graph
- ☐ Automatic skill extraction
- ☐ Trajectory mining
- ☐ Automated eval generation
- ☐ Bounded self-improvement
- ☐ Adaptive model routing
- ☐ Personalized career strategy
- ☐ Proactive opportunity discovery
- ☐ Market intelligence
- ☐ Skill-gap planning
- ☐ Multi-machine workers
- ☐ Remote sandbox
- ☐ Plugin ecosystem

---

50. First Production Milestone: Durable Verified Application

A single command should eventually guarantee this flow:

```
1. Resolve job
2. Persist job
3. Create goal/task
4. Evaluate policy
5. Evaluate fit
6. Generate tailored resume
7. Generate cover letter
8. Independent reviewer validates
9. Compile PDF
10. ATS-verify PDF
11. Request approval
12. Persist waitpoint
13. Resume after approval
14. Open browser
15. Fill application
16. Submit
17. Verify confirmation
18. Capture evidence
19. Persist outcome
20. Update memory
21. Emit trace
22. Update metrics
23. Generate improvement candidate
```

Deliberately kill the process after major steps, restart it, and verify that execution resumes without losing state or duplicating external effects.

This milestone should become the foundation of JoBot 1.0.

---

51. Clarification Questions for Final Implementation Planning

These decisions materially affect the implementation:

1. Is the target primarily:
    - personal/local tool,
    - open-source general job-search OS,
    - production desktop product,
    - future hosted SaaS,

- or general-purpose agent OS with job search as the first harness?

2. What autonomy policy should apply to final job submission?
   - always human,
   - human by default + trusted sites autonomous,
   - fully autonomous,
   - per-site/per-action policy?
3. Which platforms are genuinely Tier 1?
   - macOS
   - Linux
   - Windows
   - WSL2
   - Docker/headless
4. Should Patchright remain the default browser implementation, behind a replaceable browser capability interface?
5. Must JoBot remain local-first and self-hostable with cloud services optional?
6. Is autonomous application submission a core differentiator or merely one feature?
7. Should MCP be a first-class external interface?
8. What exact promise should JoBot 1.0 make?

Recommended 1.0 promise:

> JoBot reliably discovers, evaluates, prepares, verifies, submits, and tracks job applications across a defined set of supported boards/ATSs, with durable state, human approval, crash recovery, evidence, auditable execution, and release-gated evaluation.

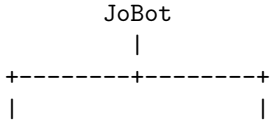
---

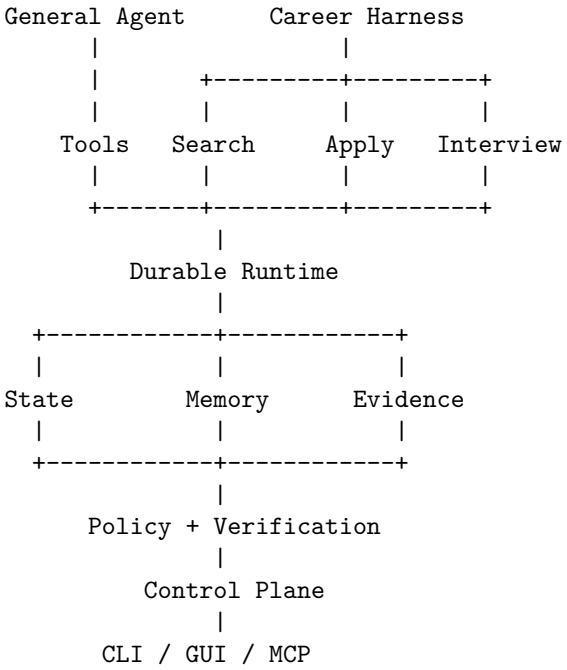
52. Final Recommendation

Do not rewrite JoBot.

Preserve the current architecture and refactor the execution substrate around durable state, events, explicit verification, effect tracking, policy, and evidence.

Then evolve the product toward:





The long-term moat is not the number of job boards JoBot supports.

It is the combination of:

durable execution + trustworthy candidate data + outcome learning + evidence + human-governed autonomy + career intelligence.

That is what can turn JoBot from an automation script into a genuine career operating system.

References and Research Sources

JoBot repository and internal artifacts

- JoBot repository: https://github.com/aryansinghnagar/JoBot
- Repository README: https://github.com/aryansinghnagar/JoBot/blob/main/README.md
- `agents.md`: https://github.com/aryansinghnagar/JoBot/blob/main/.agents/agents.md
- Merge Plan: https://github.com/aryansinghnagar/JoBot/blob/main/plan.md
- Setup Guide: https://github.com/aryansinghnagar/JoBot/blob/main/SETUP.md
- Contracts: https://github.com/aryansinghnagar/JoBot/blob/main/docs/contracts.md

Agent architecture

- LangGraph: https://github.com/langchain-ai/langgraph

- LangGraph human-in-the-loop: https://docs.langchain.com/oss/python/langchain/human-in-the-loop
- Letta: https://github.com/letta-ai/letta
- PydanticAI: https://github.com/pydantic/pydantic-ai
- OpenHands: https://github.com/All-Hands-AI/OpenHands
- Temporal: https://temporal.io/
- OpenTelemetry: https://opentelemetry.io/
- Langfuse: https://langfuse.com/
- LiteLLM: https://github.com/BerriAI/litellm
- Graphiti: https://github.com/getzep/graphiti
- Model Context Protocol: https://modelcontextprotocol.io/

Job-search and career systems

- MadsLorentzen/ai-job-search: https://github.com/MadsLorentzen/ai-job-search
- JobSpy: https://github.com/speedyapply/JobSpy
- career-ops: https://github.com/santifer/career-ops
- JobSync: https://github.com/Gsync/jobsync
- Simplify: https://simplify.jobs/
- Teal: https://www.tealhq.com/
- Jobright: https://jobright.ai/

Research positioning

The feature recommendations in this plan distinguish between: - source-derived JoBot findings, - architecture inference from the existing repository, - and comparative research into external systems.

External projects are used for architectural and product inspiration only; implementation should respect their licenses and should reimplement behavior behind JoBot's own interfaces where appropriate.